

LAPORAN PRAKTIKUM
STRUKTUR DATA
Double Linked List

disusun Oleh:

MUHAMAD FAJRI AULIA
NIM 2511532009

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T



FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
PADANG, 19 MEI 2026

KATA PENGANTAR

Puji syukur kehadiran Allah SWT yang telah memberikan rahmat, taufik, dan hidayahNya sehingga penulis dapat menyelesaikan laporan praktikum mata kuliah Struktur Data dengan baik dan tepat waktu. Laporan ini disusun untuk memenuhi salah satu tugas praktikum pada pertemuan keenam dengan judul “**Algoritma Sorting**”. Melalui penyusunan laporan ini, penulis berharap dapat memberikan gambaran mengenai Algoritma Sorting dengan java.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan laporan maupun pemahaman di masa yang akan datang. Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu serta semua pihak yang telah memberikan bimbingan, arahan, dan bantuan hingga laporan ini dapat terselesaikan. Semoga laporan ini dapat bermanfaat bagi pembaca.

Padang, 19 Mei 2026

Muhamad Fajri Aulia

DAFTAR ISI

LAPORAN PRAKTIKUM	1
KATA PENGANTAR.....	1
DAFTAR ISI.....	2
DAFTAR GAMBAR.....	3
BAB I PENDAHULUAN	4
1.1 Latar Belakang.....	4
1.2 Tujuan Praktikum	4
1.3 Persyaratan Praktikum.....	4
1.4 Waktu dan tempat pelaksanaan	5
BAB II PEMBAHASAN.....	6
2.1 Pengertian Sorting	6
2.1.1 Bubble Sort.....	6
2.1.2 Selection Sort	6
2.1.3 Insertion Sort	6
2.2 Langkah Pengerjaan	7
2.2.1 Insertion Sort	7
2.2.2 Selection Sort	8
2.2.3 Bubble Sort.....	10
2.2.4 InsertionSortGUI	12
BAB III KESIMPULAN DAN SARAN	19
3.1 Kesimpulan.....	19
3.2 Saran	19
BAB IV LAPORAN TUGAS.....	20
4.1 Kode Program	20
4.1.1 Mahasiswa_2511532009	20
4.1.2 SortingMahasiswaGUI_2511532009.....	21
4.2 Output yang dihasilkan	27
DAFTAR PUSTAKA	29

DAFTAR GAMBAR

Gambar 1.....	8
Gambar 2.....	10
Gambar 3.....	12
Gambar 4.....	18
Gambar 5.....	18
Gambar 6.....	27
Gambar 7.....	27
Gambar 8.....	28

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, pengolahan data menjadi hal yang sangat penting. Data yang tersusun secara rapi akan mempermudah proses pencarian, pengelompokan, dan pengambilan informasi. Salah satu cara untuk menyusun data tersebut adalah menggunakan algoritma sorting atau pengurutan data.

Algoritma sorting digunakan untuk mengurutkan data dari nilai terkecil ke terbesar maupun sebaliknya. Pada praktikum ini digunakan tiga metode sorting, yaitu Bubble Sort, Selection Sort, dan Insertion Sort. Ketiga metode ini merupakan algoritma dasar yang sering dipelajari oleh pemula karena mudah dipahami dan diterapkan dalam berbagai bahasa pemrograman.

Dengan mempelajari algoritma sorting, mahasiswa dapat memahami proses pengurutan data secara logika dan meningkatkan kemampuan dalam membuat program yang lebih efisien. Praktikum ini juga membantu mahasiswa memahami perbedaan cara kerja setiap algoritma sorting beserta kelebihan dan kekurangannya.

1.2 Tujuan Praktikum

- Memahami konsep dasar algoritma sorting.
- Mengetahui cara kerja Bubble Sort, Selection Sort, dan Insertion Sort.
- Mampu membuat program sorting menggunakan bahasa pemrograman Java.
- Menganalisis proses pengurutan data pada setiap algoritma sorting.

1.3 Persyaratan Praktikum

Agar praktikum berjalan lancar, mahasiswa perlu memenuhi beberapa persyaratan berikut :

- Telah mengikuti perkuliahan teori Struktur data sebagai dasar pemahaman.

- Membawa perlengkapan yang diperlukan, antara lain laptop atau computer yang sudah terpasang Java Development Kit (JDK) dan Integrated Development Enviroment (IDE) yang direkomendasikan.
- Mengikuti setiap sesi praktikum sesuai jadwal yang ditetapkan dan hadir minimal sesuai ketentuan program studi.
- Mematuhi tata tertib laboratorium, termasuk menjaga keamanan data, perangkat, serta lingkungan kerja.
- Menyusun laporan praktikum dengan format dan aturan yang telah ditetapkan dalam pedoman ini.

1.4 Waktu dan tempat pelaksanaan

Pelaksanakan praktikum Java mengikuti kalender akademik yang berlaku pada program studi. Setiap sesi praktikum dilaksanakan sesuai jadwal yang ditentukan oleh dosen pengampu. Tempat kegiatan umumnya berlangsung di laboratorium komputer, namun pada kondisi tertentu dapat dilaksanakan secara mandiri dengan perangkat masing masing, selama memenuhi syarat teknis yang ditetapkan.

BAB II

PEMBAHASAN

2.1 Pengertian Sorting

Sorting adalah proses mengurutkan data berdasarkan urutan tertentu, seperti dari kecil ke besar atau dari besar ke kecil. Sorting digunakan agar data lebih mudah dibaca, dicari, dan diproses. Dalam praktikum ini digunakan tiga metode sorting, yaitu:

2.1.1 Bubble Sort

Bubble Sort adalah metode pengurutan dengan cara membandingkan dua data yang berdekatan lalu menukarnya jika urutannya salah. Proses dilakukan berulang sampai seluruh data terurut. Dengan ciri ciri sebagai berikut :

- Mudah dipahami.
- Proses sederhana.
- Kurang efisien untuk data besar.

2.1.2 Selection Sort

Selection Sort adalah metode pengurutan dengan cara mencari nilai terkecil dari suatu bagian array lalu menukarnya dengan posisi awal. Dengan ciri sebagai berikut

- Jumlah pertukaran data lebih sedikit.
- Mudah dipelajari.
- Proses tetap lambat untuk data besar.

2.1.3 Insertion Sort

Insertion Sort adalah metode pengurutan dengan cara mengambil satu data lalu menyisipkannya ke posisi yang sesuai pada bagian data yang sudah terurut. Dengan ciri sebagai berikut:

- Cocok untuk data kecil.
- Lebih cepat dibanding Bubble Sort pada kondisi tertentu.
- Mudah diimplementasikan.

2.2 Langkah Pengerjaan

2.2.1 Insertion Sort

Kode program :

```
package pekan7_2511532009;

public class InsertSort_2511532009 {
    public static void insertionSort_2009(int[] arr_2009) {
        int n_2009 = arr_2009.length;
        for (int i_2009 = 1; i_2009 < n_2009; i_2009++) {
            int key_2009 = arr_2009[i_2009];
            int j_2009 = i_2009 - 1;
            while (j_2009 >= 0 && arr_2009[j_2009] > key_2009) {
                arr_2009[j_2009 + 1] = arr_2009[j_2009];
                j_2009--;
            }
            arr_2009[j_2009 + 1] = key_2009;
        }
    }

    public static void main(String[] args) {
        int arr_2009[] = { 23, 78, 45, 8, 32, 56, 1 };
        int n_2009 = arr_2009.length;
        System.out.printf("array yang belum terurut:\n");
        for (int i_2009 = 0; i_2009 < n_2009; i_2009++)
            System.out.print(arr_2009[i_2009] + " ");
        System.out.println("");

        insertionSort_2009(arr_2009);

        System.out.printf("array yang terurut:\n");
        for (int i_2009 = 0; i_2009 < n_2009; i_2009++)
            System.out.print(arr_2009[i_2009] + " ");
        System.out.println("");
    }
}
```

Penjelasan :

Program diawali dengan deklarasi class InsertSort_2511532009 yang digunakan sebagai tempat penyimpanan seluruh kode program. Di dalam class tersebut terdapat method insertionSort_2009 yang berfungsi untuk melakukan proses pengurutan data menggunakan algoritma Insertion Sort.

Pada method tersebut, program terlebih dahulu mengambil jumlah data pada array menggunakan arr_2009.length lalu menyimpannya ke variabel n_2009. Setelah itu program melakukan perulangan menggunakan for mulai dari indeks ke-1 karena data pertama dianggap sudah terurut.

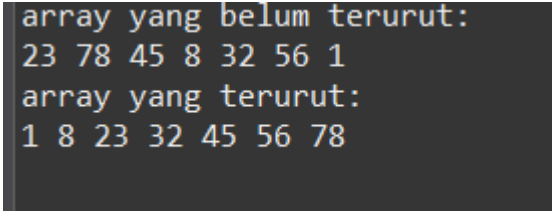
Di dalam perulangan, nilai array pada posisi tertentu disimpan ke variabel `key_2009`. Variabel ini digunakan sebagai data yang akan dibandingkan dan dipindahkan ke posisi yang sesuai. Selanjutnya variabel `j_2009` digunakan untuk memeriksa data sebelumnya.

Proses utama Insertion Sort terjadi pada bagian `while`, yaitu saat program membandingkan data sebelumnya dengan `key_2009`. Jika data sebelumnya lebih besar, maka data akan digeser ke kanan sampai ditemukan posisi yang tepat. Setelah proses pergeseran selesai, nilai `key_2009` ditempatkan pada posisi yang benar sehingga data menjadi lebih teratur.

Pada method `main`, program membuat array dengan isi data 23, 78, 45, 8, 32, 56, 1. Kemudian program menampilkan isi array sebelum diurutkan menggunakan perulangan `for`. Setelah itu method `insertionSort_2009` dipanggil untuk menjalankan proses pengurutan data.

Setelah proses sorting selesai, program kembali menggunakan perulangan untuk menampilkan hasil array yang sudah teratur. Hasil akhirnya adalah 1 8 23 32 45 56 78.

Output yang dihasilkan adalah sebagai berikut :



```
array yang belum teratur:
23 78 45 8 32 56 1
array yang teratur:
1 8 23 32 45 56 78
```

Gambar 1

2.2.2 Selection Sort

Kode program :

```
package pekan7_2511532009;
public class SelectionSort_2511532009 {
    public static void selectionSort_2511532009 (int [] arr_2009) {
        int n_2009 = arr_2009.length;
        for (int i_2009 = 0; i_2009 < n_2009 ; i_2009++) {
            int minIndex_2009 = i_2009;
            for (int j_2009 = i_2009 +1; j_2009 < n_2009 ; j_2009++) {
                if (arr_2009 [j_2009]< arr_2009[minIndex_2009]) {
                    minIndex_2009 = j_2009;
                }
            }
            int temp_2009 = arr_2009 [i_2009];
```

```

        arr_2009 [i_2009] = arr_2009 [minIndex_2009];
        arr_2009 [minIndex_2009] = temp_2009;
    }
}

public static void main (String [] args) {
    int arr_2009 [] = {23, 78, 45, 8, 32, 56, 1};
    int n_2009 = arr_2009.length;
    System.out.printf("array yang belum terurut:\n");
    for (int i_2009 = 0; i_2009 < n_2009 ; i_2009++)
        System.out.print(arr_2009[i_2009] + " ");
    System.out.println("");
    selectionSort_2511532009(arr_2009);
    System.out.printf("array yang terurut:\n");
    for (int i_2009 = 0; i_2009 < n_2009 ; i_2009++)
        System.out.print(arr_2009 [i_2009] + " ");
    System.out.println("");
}
}

```

Penjelasan :

Program diawali dengan deklarasi class SelectionSort_2511532009 yang digunakan sebagai tempat penyimpanan seluruh kode program. Di dalam class tersebut terdapat method selectionSort_2511532009 yang berfungsi untuk mengurutkan data menggunakan algoritma Selection Sort.

Pada method tersebut, program terlebih dahulu mengambil jumlah data dalam array menggunakan arr_2009.length lalu menyimpannya ke variabel n_2009. Setelah itu program menjalankan perulangan for untuk memeriksa setiap elemen array satu per satu.

Variabel minIndex_2009 digunakan untuk menyimpan posisi indeks dengan nilai terkecil. Awalnya nilai minIndex_2009 diisi dengan indeks saat ini yaitu i_2009. Kemudian program menjalankan perulangan kedua menggunakan j_2009 untuk mencari nilai yang lebih kecil dari posisi awal.

Pada bagian if, program membandingkan setiap data pada array. Jika ditemukan nilai yang lebih kecil dari nilai pada minIndex_2009, maka posisi minIndex_2009 akan diperbarui sesuai indeks data terkecil tersebut.

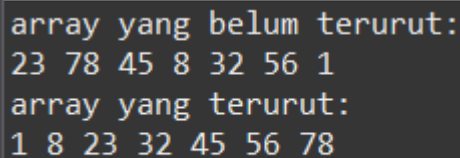
Setelah proses pencarian selesai, program melakukan pertukaran data menggunakan variabel sementara temp_2009. Nilai pada indeks awal disimpan sementara ke dalam temp_2009, kemudian nilai terkecil dipindahkan ke posisi

awal, lalu nilai lama dikembalikan ke posisi sebelumnya. Proses ini dilakukan terus menerus sampai seluruh data terurut.

Pada method main, program membuat array berisi data 23, 78, 45, 8, 32, 56, 1. Setelah itu program menampilkan isi array sebelum diurutkan menggunakan perulangan for.

Kemudian method `selectionSort_2511532009` dipanggil untuk menjalankan proses pengurutan data menggunakan algoritma Selection Sort. Setelah proses sorting selesai, program kembali menampilkan isi array yang sudah terurut.

Output yang dihasilkan adalah sebagai berikut:



```
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78
```

Gambar 2

2.2.3 Bubble Sort

Kode program :

```
package pekan7_2511532009;

public class BubbleSort_2511532009 {
    public static void bubbleSort_2009(int[] arr_2009) {
        int n_2009 = arr_2009.length;
        for (int i_2009 = 0; i_2009 < n_2009; i_2009++) {
            for (int j_2009 = 0; j_2009 < n_2009 - i_2009 - 1; j_2009++) {
                if (arr_2009[j_2009] > arr_2009[j_2009 + 1]) {
                    int temp_2009 = arr_2009[j_2009];
                    arr_2009[j_2009] = arr_2009[j_2009 + 1];
                    arr_2009[j_2009 + 1] = temp_2009;
                    // System.out.println("data:" + arr_2009[j_2009] + " " +
arr_2009[j_2009 + 1]);
                }
            }
        }
    }

    public static void main(String[] args) {
        int arr_2009[] = { 23, 78, 45, 8, 32, 56, 1 };
        int n_2009 = arr_2009.length;
        System.out.print("array yang belum terurut: ");
    }
}
```

```

for (int i_2009 = 0; i_2009 < n_2009; i_2009++)
    System.out.print(arr_2009[i_2009] + " ");
System.out.println("");

bubbleSort_2009(arr_2009);

System.out.print("array yang terurut menggunakan BubbleSort: ");
for (int i_2009 = 0; i_2009 < n_2009; i_2009++)
    System.out.print(arr_2009[i_2009] + " ");
System.out.println("");
}
}

```

Penjelasan :

Program diawali dengan deklarasi class BubbleSort_2511532009 yang digunakan sebagai tempat penyimpanan seluruh kode program. Di dalam class tersebut terdapat method bubbleSort_2009 yang berfungsi untuk mengurutkan data menggunakan algoritma Bubble Sort.

Pada method tersebut, program terlebih dahulu mengambil jumlah data dalam array menggunakan arr_2009.length lalu menyimpannya ke variabel n_2009. Setelah itu program menjalankan perulangan pertama menggunakan for yang berfungsi untuk mengatur jumlah proses pengurutan.

Di dalam perulangan pertama terdapat perulangan kedua yang digunakan untuk membandingkan dua data yang berdekatan. Program akan memeriksa apakah nilai pada indeks saat ini lebih besar dibanding nilai setelahnya.

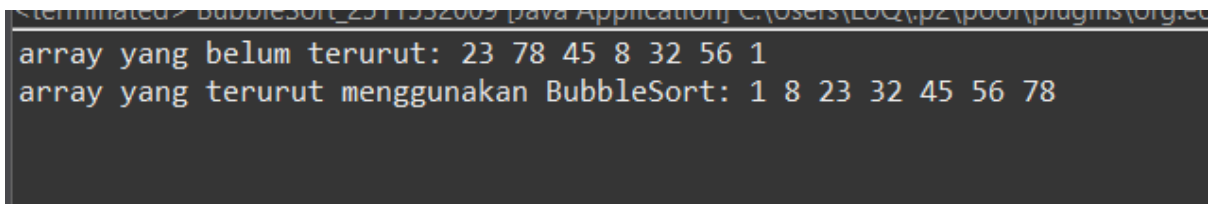
Pada bagian if, jika data sebelah kiri lebih besar daripada data sebelah kanan, maka kedua data akan ditukar posisinya. Proses pertukaran dilakukan menggunakan variabel sementara temp_2009. Nilai sementara disimpan terlebih dahulu agar data tidak hilang saat proses pertukaran berlangsung.

Proses perbandingan dan pertukaran dilakukan secara terus menerus sampai seluruh data tersusun dari nilai terkecil ke terbesar. Pada setiap perulangan, nilai terbesar akan berpindah ke bagian akhir array sehingga disebut Bubble Sort karena prosesnya seperti gelembung yang naik ke atas.

Pada method main, program membuat array berisi data 23, 78, 45, 8, 32, 56, 1. Setelah itu program menampilkan isi array sebelum diurutkan menggunakan perulangan for.

Kemudian method bubbleSort_2009 dipanggil untuk menjalankan proses pengurutan data menggunakan algoritma Bubble Sort. Setelah proses sorting selesai, program kembali menampilkan isi array yang sudah terurut.

Output yang dihasilkan adalah sebagai berikut :



```
<terminated> BubbleSort_2511532009 [Java Application] C:\Users\LOQ\AppData\Local\Temp\plugins\org.ej
array yang belum terurut: 23 78 45 8 32 56 1
array yang terurut menggunakan BubbleSort: 1 8 23 32 45 56 78
```

Gambar 3

2.2.4 InsertionSortGUI

Kode Program :

```
package pekan7_2511532009;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;

public class InsertionSortGUI_2511532009 extends JFrame {

    private static final long serialVersionUID = 1L;
    private JPanel contentPane_2009;
```

```

private int[] array_2009;
private JLabel[] labelArray_2009;
private JButton stepButton_2009, resetButton_2009, setButton_2009;
private JTextField inputField_2009;
private JPanel panelArray_2009;
private JTextArea stepArea_2009;

private int i_2009 = 1, j_2009;
private boolean sorting_2009 = false;
private int stepCount_2009 = 1;

public InsertionSortGUI_2511532009() {
    setTitle("Insertion Sort Langkah per Langkah");
    setSize(750, 400);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout());

    // Panel input
    JPanel inputPanel_2009 = new JPanel(new FlowLayout());
    inputField_2009 = new JTextField(30);
    setButton_2009 = new JButton("Set Array");
    inputPanel_2009.add(new JLabel("Masukkan angka (pisahkan dengan
koma):"));
    inputPanel_2009.add(inputField_2009);
    inputPanel_2009.add(setButton_2009);

    // Panel array visual
    panelArray_2009 = new JPanel();
    panelArray_2009.setLayout(new FlowLayout());

    // Panel kontrol
    JPanel controlPanel_2009 = new JPanel();
    stepButton_2009 = new JButton("Langkah Selanjutnya");
    resetButton_2009 = new JButton("Reset");
    stepButton_2009.setEnabled(false);
    controlPanel_2009.add(stepButton_2009);
    controlPanel_2009.add(resetButton_2009);

    // Area teks untuk log langkah-langkah
    stepArea_2009 = new JTextArea(8, 60);
    stepArea_2009.setEditable(false);
    stepArea_2009.setFont(new Font("Monospaced", Font.PLAIN, 14));
    JScrollPane scrollPane_2009 = new JScrollPane(stepArea_2009);

    // Tambahkan panel ke frame
    add(inputPanel_2009, BorderLayout.NORTH);
    add(panelArray_2009, BorderLayout.CENTER);

```

```

add(controlPanel_2009, BorderLayout.SOUTH);
add(scrollPane_2009, BorderLayout.EAST);

// Event Set Array
setButton_2009.addActionListener(e -> setArrayFromInput());

// Event Langkah Selanjutnya
stepButton_2009.addActionListener(e -> performStep_2009());

// Event Reset
resetButton_2009.addActionListener(e -> reset_2009());
}

private void setArrayFromInput() {
    String text_2009 = inputField_2009.getText().trim();
    if (text_2009.isEmpty()) return;
    String[] parts = text_2009.split(",");
    array_2009 = new int[parts.length];
    try {
        for (int k_2009 = 0; k_2009 < parts.length; k_2009++) {
            array_2009[k_2009] = Integer.parseInt(parts[k_2009].trim());
        }
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang
dipisahkan "
        + "dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }
    i_2009 = 1;
    stepCount_2009 = 1;
    sorting_2009 = true;
    stepButton_2009.setEnabled(true);
    stepArea_2009.setText("");
    panelArray_2009.removeAll();
    labelArray_2009 = new JLabel[array_2009.length];
    for (int k_2009 = 0; k_2009 < array_2009.length; k_2009++) {
        labelArray_2009[k_2009] = new
JLabel(String.valueOf(array_2009[k_2009]));
        labelArray_2009[k_2009].setFont(new Font("Arial", Font.BOLD, 24));
        labelArray_2009[k_2009].setBorder(BorderFactory.createLineBorder(C
olor.BLACK));
        labelArray_2009[k_2009].setPreferredSize(new Dimension(50, 50));
        labelArray_2009[k_2009].setHorizontalAlignment(SwingConstants.CENT
ER);
        panelArray_2009.add(labelArray_2009[k_2009]);
    }
    panelArray_2009.revalidate();
    panelArray_2009.repaint();
}

```

```

    }

    private void performStep_2009() {
        if (i_2009 < array_2009.length && sorting_2009) {
            int key_2009 = array_2009[i_2009];
            j_2009 = i_2009 - 1;

            StringBuilder stepLog_2009 = new StringBuilder();
            stepLog_2009.append("Langkah ").append(stepCount_2009).
            append(": Memasukkan ").append(key_2009).append("\n");

            while (j_2009 >= 0 && array_2009[j_2009] > key_2009) {
                array_2009[j_2009 + 1] = array_2009[j_2009];
                j_2009--;
            }
            array_2009[j_2009 + 1] = key_2009;

            updateLabels_2009();
            stepLog_2009.append("Hasil:
").append(arrayToString_2009(array_2009)).append("\n\n");
            stepArea_2009.append(stepLog_2009.toString());

            i_2009++;
            stepCount_2009++;

            if (i_2009 == array_2009.length) {
                sorting_2009 = false;
                stepButton_2009.setEnabled(false);
                JOptionPane.showMessageDialog(this, "Sorting selesai!");
            }
        }
    }

    private void updateLabels_2009() {
        for (int k_2009 = 0; k_2009 < array_2009.length; k_2009++) {
            labelArray_2009[k_2009].setText(String.valueOf(array_2009[k_2009])
);
        }
    }

    private void reset_2009() {
        inputField_2009.setText("");
        panelArray_2009.removeAll();
        panelArray_2009.revalidate();
        panelArray_2009.repaint();
        stepArea_2009.setText("");
        stepButton_2009.setEnabled(false);
        sorting_2009 = false;
    }

```



```

        i_2009 = 1;
        stepCount_2009 = 1;
    }

    private String arrayToString_2009(int[] arr) {
        StringBuilder sb_2009 = new StringBuilder();
        for (int k_2009 = 0; k_2009 < arr.length; k_2009++) {
            sb_2009.append(arr[k_2009]);
            if (k_2009 < arr.length - 1) sb_2009.append(", ");
        }
        return sb_2009.toString();
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            InsertionSortGUI_2511532009 gui_2009 = new
InsertionSortGUI_2511532009();
            gui_2009.setVisible(true);
        });
    }
}

```

Penjelasan :

Program diawali dengan deklarasi class InsertionSortGUI_2511532009 yang merupakan turunan dari JFrame. Class ini digunakan untuk membuat tampilan GUI atau Graphical User Interface sehingga proses Insertion Sort dapat dilihat secara visual dan langkah demi langkah.

Pada bagian awal program terdapat beberapa import library dari Java Swing dan AWT seperti JFrame, JButton, JLabel, JPanel, JTextField, dan JTextArea.

Library tersebut digunakan untuk membuat tampilan aplikasi seperti tombol, kotak input, area teks, dan panel.

Program memiliki beberapa variabel penting seperti array_2009 untuk menyimpan data array, labelArray_2009 untuk menampilkan data array pada GUI, serta tombol stepButton_2009, resetButton_2009, dan setButton_2009 untuk mengontrol jalannya program.

Pada constructor InsertionSortGUI_2511532009(), program mengatur tampilan frame seperti judul aplikasi, ukuran window, posisi tampilan, dan layout. Setelah

itu program membuat panel input yang digunakan untuk memasukkan angka dengan format dipisahkan menggunakan koma.

Program juga membuat panel visual array yang digunakan untuk menampilkan isi array dalam bentuk kotak-kotak angka. Selain itu terdapat panel kontrol yang berisi tombol “Langkah Selanjutnya” dan “Reset”.

Bagian `stepArea_2009` digunakan untuk menampilkan proses langkah-langkah sorting dalam bentuk teks. Area ini membantu pengguna melihat perubahan data pada setiap langkah pengurutan.

Pada event `setButton_2009.addActionListener`, program menjalankan method `setArrayFromInput()` saat tombol “Set Array” ditekan. Method ini berfungsi membaca input angka dari pengguna, memisahkan data menggunakan koma, lalu mengubahnya menjadi array integer.

Jika pengguna memasukkan data yang bukan angka, program akan menampilkan pesan error menggunakan `JOptionPane`. Setelah data berhasil dimasukkan, program membuat label-label angka pada panel array agar data dapat ditampilkan secara visual.

Method `performStep_2009()` digunakan untuk menjalankan satu langkah proses Insertion Sort setiap kali tombol “Langkah Selanjutnya” ditekan. Program mengambil nilai `key_2009`, lalu membandingkannya dengan data sebelumnya. Jika data sebelumnya lebih besar, maka data akan digeser ke kanan sampai ditemukan posisi yang sesuai.

Setelah proses selesai, program memperbarui tampilan array menggunakan method `updateLabels_2009()`. Method ini berfungsi mengubah isi label sesuai data array terbaru setelah sorting dilakukan.

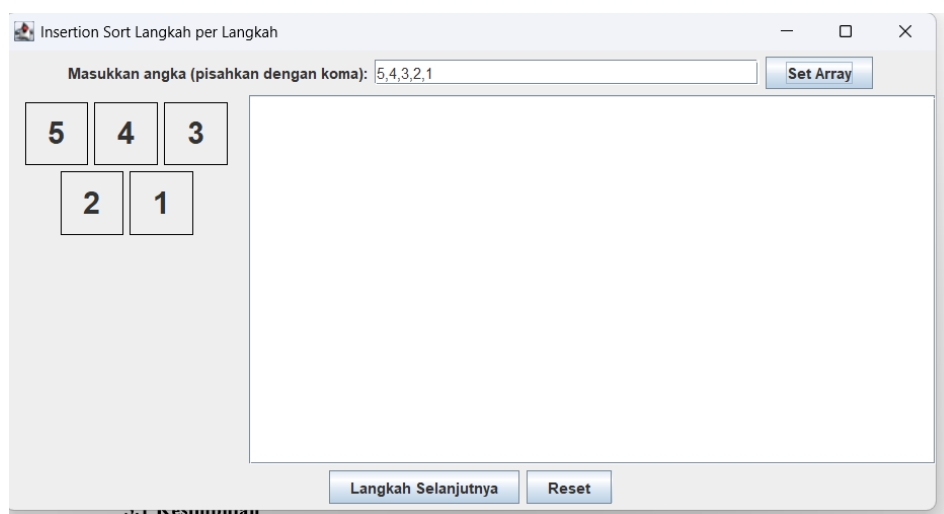
Program juga menampilkan log langkah sorting pada `stepArea_2009`. Setiap langkah akan menampilkan angka yang diproses serta hasil array setelah pengurutan dilakukan.

Method `reset_2009()` digunakan untuk mengembalikan tampilan program ke kondisi awal. Method ini akan menghapus input, membersihkan panel array, menghapus log langkah, dan mengatur ulang variabel sorting.

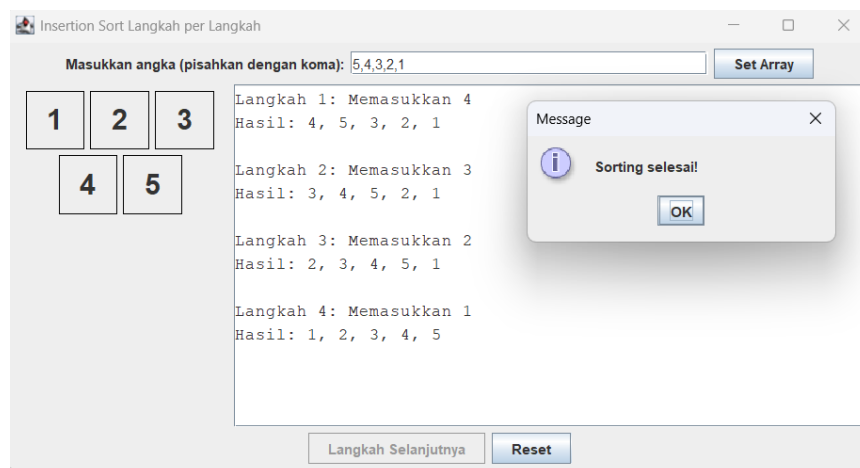
Method `arrayToString_2009()` digunakan untuk mengubah isi array menjadi bentuk string agar dapat ditampilkan pada area teks langkah-langkah sorting.

Pada method `main`, program menjalankan GUI menggunakan `SwingUtilities.invokeLater()`. Setelah program dijalankan, pengguna dapat memasukkan angka, melihat proses Insertion Sort langkah demi langkah, dan memahami cara kerja algoritma sorting secara visual.

Output yang dihasilkan adalah sebagai berikut :



Gambar 4



Gambar 5

BAB III

KESIMPULAN DAN SARAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma sorting digunakan untuk mengurutkan data agar lebih terstruktur dan mudah diproses. Bubble Sort, Selection Sort, dan Insertion Sort memiliki cara kerja yang berbeda dalam melakukan pengurutan data. Bubble Sort bekerja dengan membandingkan data yang berdekatan. Selection Sort bekerja dengan mencari nilai terkecil atau terbesar lalu menukarnya dengan posisi tertentu. Insertion Sort bekerja dengan menyisipkan data ke posisi yang sesuai. Dengan praktikum ini, mahasiswa dapat memahami konsep dasar sorting serta mampu mengimplementasikan algoritma pengurutan data menggunakan bahasa pemrograman Java.

3.2 Saran

- Praktikan sebaiknya lebih sering berlatih membuat program sorting agar memahami alur algoritma dengan baik.
- Praktikan perlu mencoba berbagai jenis data untuk melihat perbedaan hasil dan proses sorting.
- Praktikan disarankan mempelajari algoritma sorting lain agar wawasan pemrograman menjadi lebih luas.

BAB IV LAPORAN TUGAS

4.1 Kode Program

4.1.1 Mahasiswa_2511532009

```
package pekan7_2511532009;

// Kelas ADT untuk menyimpan data mahasiswa
public class Mahasiswa_2511532009 {

    // Atribut data mahasiswa
    private String nama_2009;
    private String nim_2009;
    private String prodi_2009;

    // Constructor
    public Mahasiswa_2511532009(String nama_2009, String nim_2009, String
prodi_2009) {
        this.nama_2009 = nama_2009;
        this.nim_2009 = nim_2009;
        this.prodi_2009 = prodi_2009;
    }

    // Getter dan Setter
    public String getNama_2009() {
        return nama_2009;
    }

    public void setNama_2009(String nama_2009) {
        this.nama_2009 = nama_2009;
    }

    public String getNim_2009() {
        return nim_2009;
    }

    public void setNim_2009(String nim_2009) {
        this.nim_2009 = nim_2009;
    }

    public String getProdi_2009() {
        return prodi_2009;
    }

    public void setProdi_2009(String prodi_2009) {
        this.prodi_2009 = prodi_2009;
    }

    // toString untuk menampilkan data mahasiswa
    @Override
    public String toString() {
        return nama_2009 + " | " + nim_2009 + " | " + prodi_2009;
    }
}
```

4.1.2 SortingMahasiswaGUI_2511532009

```
package pekan7_2511532009;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.GridLayout;
import java.util.ArrayList;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTable;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.table.DefaultTableModel;

public class SortingMahasiswaGUI_2511532009 extends JFrame {

    private static final long serialVersionUID = 1L;

    // Data mahasiswa
    private ArrayList<Mahasiswa_2511532009> listMahasiswa_2009 = new
ArrayList<>();

    // Komponen input
    private JTextField fieldNama_2009, fieldNim_2009, fieldProdi_2009;
    private JButton btnTambah_2009, btnHapus_2009, btnMulaiSorting_2009,
btnReset_2009;
    private JComboBox<String> comboAlgoritma_2009;

    // Tabel data
    private JTable tableMahasiswa_2009;
    private DefaultTableModel tableModel_2009;

    // Area visualisasi sorting
    private JTextArea areaVisualisasi_2009;

    public SortingMahasiswaGUI_2511532009() {
        setTitle("Sorting Nama Mahasiswa - Pekan 7");
        setSize(900, 650);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout(10, 10));

        // ===== PANEL INPUT (NORTH) =====
        JPanel panelInput_2009 = new JPanel(new GridLayout(4, 2, 5, 5));
        panelInput_2009.setBorder(BorderFactory.createTitledBorder("Input Data
Mahasiswa"));
    }
}
```

```

fieldNama_2009 = new JTextField();
fieldNim_2009 = new JTextField();
fieldProdi_2009 = new JTextField();

panelInput_2009.add(new JLabel(" Nama Mahasiswa :"));
panelInput_2009.add(fieldNama_2009);
panelInput_2009.add(new JLabel(" NIM :"));
panelInput_2009.add(fieldNim_2009);
panelInput_2009.add(new JLabel(" Program Studi :"));
panelInput_2009.add(fieldProdi_2009);

// Tombol tambah dan hapus
JPanel panelTombolInput_2009 = new JPanel(new
FlowLayout(FlowLayout.LEFT));
btnTambah_2009 = new JButton("Tambah Data");
btnHapus_2009 = new JButton("Hapus Data");
btnTambah_2009.setBackground(new Color(70, 130, 180));
btnTambah_2009.setForeground(Color.WHITE);
btnHapus_2009.setBackground(new Color(220, 80, 80));
btnHapus_2009.setForeground(Color.WHITE);
panelTombolInput_2009.add(btnTambah_2009);
panelTombolInput_2009.add(btnHapus_2009);
panelInput_2009.add(new JLabel(""));
panelInput_2009.add(panelTombolInput_2009);

// ===== PANEL TABEL (CENTER-LEFT) =====
String[] kolomHeader_2009 = {"Nama Mahasiswa", "NIM", "Program Studi"};
tableModel_2009 = new DefaultTableModel(kolomHeader_2009, 0);
tableMahasiswa_2009 = new JTable(tableModel_2009);
tableMahasiswa_2009.setFont(new Font("Arial", Font.PLAIN, 13));
tableMahasiswa_2009.setRowHeight(25);
tableMahasiswa_2009.getTableHeader().setFont(new Font("Arial", Font.BOLD,
13));
JScrollPane scrollTabel_2009 = new JScrollPane(tableMahasiswa_2009);
scrollTabel_2009.setBorder(BorderFactory.createTitledBorder("Daftar
Mahasiswa"));
scrollTabel_2009.setPreferredSize(new Dimension(420, 250));

// ===== PANEL SORTING (CENTER-RIGHT)
=====
JPanel panelSorting_2009 = new JPanel(new BorderLayout(5, 5));
panelSorting_2009.setBorder(BorderFactory.createTitledBorder("Pilih
Algoritma Sorting"));

String[] algoritmaList_2009 = {"Insertion Sort", "Selection Sort", "Bubble
Sort"};
comboAlgoritma_2009 = new JComboBox<>(algoritmaList_2009);
comboAlgoritma_2009.setFont(new Font("Arial", Font.PLAIN, 13));

JPanel panelKombo_2009 = new JPanel(new FlowLayout(FlowLayout.LEFT));
panelKombo_2009.add(new JLabel("Algoritma :"));
panelKombo_2009.add(comboAlgoritma_2009);

btnMulaiSorting_2009 = new JButton("Mulai Sorting");
btnReset_2009 = new JButton("Reset Visualisasi");
btnMulaiSorting_2009.setBackground(new Color(60, 170, 90));
btnMulaiSorting_2009.setForeground(Color.WHITE);
btnReset_2009.setBackground(new Color(150, 150, 150));

```

```

        btnReset_2009.setForeground(Color.WHITE);
        btnMulaiSorting_2009.setFont(new Font("Arial", Font.BOLD, 13));

        JPanel panelTombolSort_2009 = new JPanel(new FlowLayout(FlowLayout.LEFT));
        panelTombolSort_2009.add(btnMulaiSorting_2009);
        panelTombolSort_2009.add(btnReset_2009);

        panelSorting_2009.add(panelKombo_2009, BorderLayout.NORTH);
        panelSorting_2009.add(panelTombolSort_2009, BorderLayout.SOUTH);

        // ===== PANEL TENGAH =====
        JPanel panelTengah_2009 = new JPanel(new BorderLayout(10, 0));
        panelTengah_2009.add(scrollTabel_2009, BorderLayout.CENTER);
        panelTengah_2009.add(panelSorting_2009, BorderLayout.EAST);

        // ===== PANEL VISUALISASI (SOUTH) =====
        areaVisualisasi_2009 = new JTextArea(12, 70);
        areaVisualisasi_2009.setEditable(false);
        areaVisualisasi_2009.setFont(new Font("Monospaced", Font.PLAIN, 13));
        areaVisualisasi_2009.setBackground(new Color(245, 245, 245));
        JScrollPane scrollVisualisasi_2009 = new
JScrollPane(areaVisualisasi_2009);

scrollVisualisasi_2009.setBorder(BorderFactory.createTitledBorder("Visualisasi
Proses Sorting"));

        // ===== TAMBAHKAN KE FRAME =====
        add(panelInput_2009, BorderLayout.NORTH);
        add(panelTengah_2009, BorderLayout.CENTER);
        add(scrollVisualisasi_2009, BorderLayout.SOUTH);

        // ===== EVENT LISTENER =====
        btnTambah_2009.addActionListener(e -> tambahData_2009());
        btnHapus_2009.addActionListener(e -> hapusData_2009());
        btnMulaiSorting_2009.addActionListener(e -> mulaiSorting_2009());
        btnReset_2009.addActionListener(e -> resetVisualisasi_2009());
    }

    // Tambah data mahasiswa ke tabel dan list
    private void tambahData_2009() {
        String nama_2009 = fieldNama_2009.getText().trim();
        String nim_2009 = fieldNim_2009.getText().trim();
        String prodi_2009 = fieldProdi_2009.getText().trim();

        if (nama_2009.isEmpty() || nim_2009.isEmpty() || prodi_2009.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Semua field harus diisi!",
"Peringatan",
                JOptionPane.WARNING_MESSAGE);
            return;
        }

        Mahasiswa_2511532009 mhs_2009 = new Mahasiswa_2511532009(nama_2009,
nim_2009, prodi_2009);
        listMahasiswa_2009.add(mhs_2009);
        tableModel_2009.addRow(new Object[]{nama_2009, nim_2009, prodi_2009});

        // Bersihkan field setelah tambah
        fieldNama_2009.setText("");
        fieldNim_2009.setText("");

```



```

        fieldProdi_2009.setText("");
        fieldNama_2009.requestFocus();
    }

    // Hapus data mahasiswa yang dipilih dari tabel
    private void hapusData_2009() {
        int baris_2009 = tableMahasiswa_2009.getSelectedRow();
        if (baris_2009 == -1) {
            JOptionPane.showMessageDialog(this, "Pilih data yang ingin dihapus!",
"Peringatan",
                JOptionPane.WARNING_MESSAGE);
            return;
        }
        listMahasiswa_2009.remove(baris_2009);
        tableModel_2009.removeRow(baris_2009);
    }

    // Reset area visualisasi
    private void resetVisualisasi_2009() {
        areaVisualisasi_2009.setText("");
    }

    // Mulai sorting sesuai algoritma yang dipilih
    private void mulaiSorting_2009() {
        if (listMahasiswa_2009.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Belum ada data mahasiswa!",
"Peringatan",
                JOptionPane.WARNING_MESSAGE);
            return;
        }
    }

    // Salin data awal (identik untuk ketiga algoritma)
    ArrayList<Mahasiswa_2511532009> dataSorting_2009 =
salinData_2009(listMahasiswa_2009);
    String pilihan_2009 = (String) comboAlgoritma_2009.getSelectedItem();

    areaVisualisasi_2009.setText("");

    // Jalankan algoritma sesuai pilihan
    if ("Insertion Sort".equals(pilihan_2009)) {
        insertionSort_2009(dataSorting_2009);
    } else if ("Selection Sort".equals(pilihan_2009)) {
        selectionSort_2009(dataSorting_2009);
    } else if ("Bubble Sort".equals(pilihan_2009)) {
        bubbleSort_2009(dataSorting_2009);
    }
}

// Salin ArrayList agar data awal tidak berubah
private ArrayList<Mahasiswa_2511532009>
salinData_2009(ArrayList<Mahasiswa_2511532009> sumber_2009) {
    ArrayList<Mahasiswa_2511532009> salinan_2009 = new ArrayList<>();
    for (Mahasiswa_2511532009 mhs_2009 : sumber_2009) {
        salinan_2009.add(new Mahasiswa_2511532009(
            mhs_2009.getNama_2009(),
            mhs_2009.getNim_2009(),
            mhs_2009.getProdi_2009()
        ));
    }
}

```

```

        return salinan_2009;
    }

    // Konversi list ke string nama untuk ditampilkan
    private String listKeString_2009(ArrayList<Mahasiswa_2511532009> list_2009) {
        StringBuilder sb_2009 = new StringBuilder("");
        for (int k_2009 = 0; k_2009 < list_2009.size(); k_2009++) {
            sb_2009.append(list_2009.get(k_2009).getNama_2009());
            if (k_2009 < list_2009.size() - 1) sb_2009.append(", ");
        }
        sb_2009.append("]");
        return sb_2009.toString();
    }

    // ===== INSERTION SORT =====
    private void insertionSort_2009(ArrayList<Mahasiswa_2511532009> arr_2009) {
        int n_2009 = arr_2009.size();
        StringBuilder log_2009 = new StringBuilder();
        log_2009.append("=== INSERTION SORT ===\n");
        log_2009.append("Data awal :");
        ").append(listKeString_2009(arr_2009)).append("\n\n");

        // Proses sorting insertion sort
        for (int i_2009 = 1; i_2009 < n_2009; i_2009++) {
            Mahasiswa_2511532009 key_2009 = arr_2009.get(i_2009);
            int j_2009 = i_2009 - 1;

            // Geser elemen yang lebih besar dari key ke kanan
            while (j_2009 >= 0 && arr_2009.get(j_2009).getNama_2009()
                .compareToIgnoreCase(key_2009.getNama_2009()) > 0) {
                arr_2009.set(j_2009 + 1, arr_2009.get(j_2009));
                j_2009--;
            }
            arr_2009.set(j_2009 + 1, key_2009);

            // Tampilkan hasil tiap langkah
            log_2009.append("Langkah ").append(i_2009)
                .append(" :");
            ").append(listKeString_2009(arr_2009)).append("\n");
        }

        log_2009.append("\nHasil Akhir :");
        ").append(listKeString_2009(arr_2009)).append("\n");
        areaVisualisasi_2009.setText(log_2009.toString());
    }

    // ===== SELECTION SORT =====
    private void selectionSort_2009(ArrayList<Mahasiswa_2511532009> arr_2009) {
        int n_2009 = arr_2009.size();
        StringBuilder log_2009 = new StringBuilder();
        log_2009.append("=== SELECTION SORT ===\n");
        log_2009.append("Data awal :");
        ").append(listKeString_2009(arr_2009)).append("\n\n");

        // Proses sorting selection sort
        for (int i_2009 = 0; i_2009 < n_2009 - 1; i_2009++) {
            int minIndex_2009 = i_2009;

            // Cari index minimum dari sisa elemen

```

```

        for (int j_2009 = i_2009 + 1; j_2009 < n_2009; j_2009++) {
            if (arr_2009.get(j_2009).getNama_2009()
                .compareToIgnoreCase(arr_2009.get(minIndex_2009).getNama_2
009()) < 0) {
                minIndex_2009 = j_2009;
            }
        }

        // Tukar elemen minimum dengan posisi i
        Mahasiswa_2511532009 temp_2009 = arr_2009.get(i_2009);
        arr_2009.set(i_2009, arr_2009.get(minIndex_2009));
        arr_2009.set(minIndex_2009, temp_2009);

        // Tampilkan hasil tiap pass
        log_2009.append("Pass ").append(i_2009 + 1)
            .append(" :
").append(listKeString_2009(arr_2009)).append("\n");
    }

    log_2009.append("\nHasil Akhir :
").append(listKeString_2009(arr_2009)).append("\n");
    areaVisualisasi_2009.setText(log_2009.toString());
}

// ===== BUBBLE SORT =====
private void bubbleSort_2009(ArrayList<Mahasiswa_2511532009> arr_2009) {
    int n_2009 = arr_2009.size();
    StringBuilder log_2009 = new StringBuilder();
    log_2009.append("=== BUBBLE SORT ===\n");
    log_2009.append("Data awal :
").append(listKeString_2009(arr_2009)).append("\n\n");

    // Proses sorting bubble sort
    for (int i_2009 = 0; i_2009 < n_2009 - 1; i_2009++) {
        boolean adaSwap_2009 = false;

        // Bandingkan dan tukar elemen yang berdekatan
        for (int j_2009 = 0; j_2009 < n_2009 - i_2009 - 1; j_2009++) {
            if (arr_2009.get(j_2009).getNama_2009()
                .compareToIgnoreCase(arr_2009.get(j_2009 +
1).getNama_2009()) > 0) {
                Mahasiswa_2511532009 temp_2009 = arr_2009.get(j_2009);
                arr_2009.set(j_2009, arr_2009.get(j_2009 + 1));
                arr_2009.set(j_2009 + 1, temp_2009);
                adaSwap_2009 = true;
            }
        }
    }

    // Tampilkan hasil tiap pass
    log_2009.append("Pass ").append(i_2009 + 1)
        .append(" :
").append(listKeString_2009(arr_2009)).append("\n");

    // Jika tidak ada swap, array sudah terurut
    if (!adaSwap_2009) break;
}

    log_2009.append("\nHasil Akhir :
").append(listKeString_2009(arr_2009)).append("\n");

```

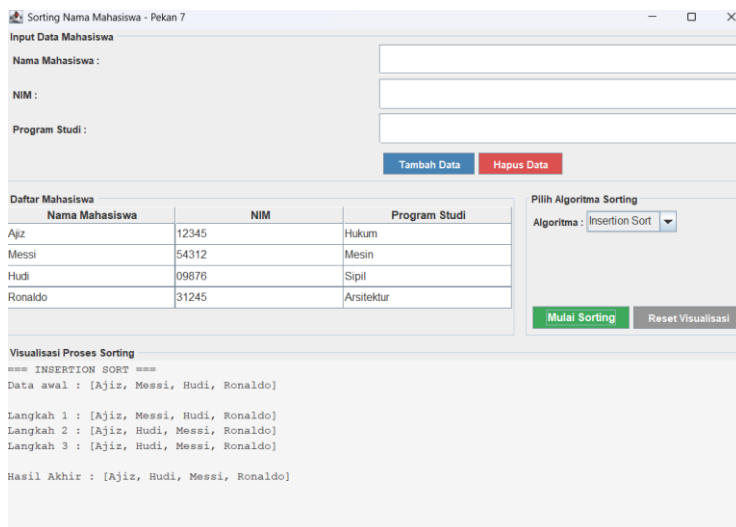
```

        areaVisualisasi_2009.setText(log_2009.toString());
    }

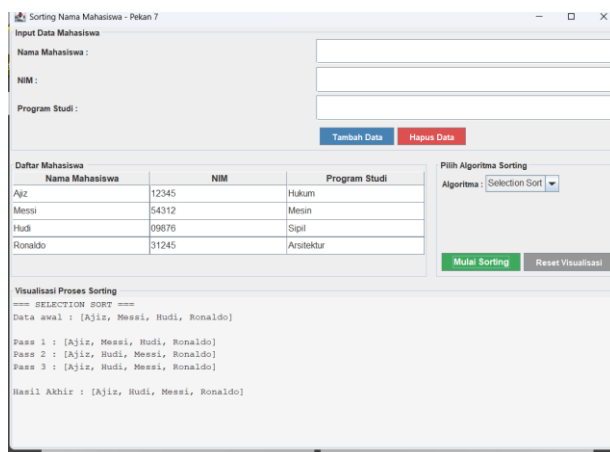
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            SortingMahasiswaGUI_2511532009 gui_2009 = new
SortingMahasiswaGUI_2511532009();
            gui_2009.setVisible(true);
        });
    }
}

```

4.2 Output yang dihasilkan



Gambar 6



Gambar 7

Input Data Mahasiswa

Nama Mahasiswa :

NIM :

Program Studi :

Tambah Data

Hapus Data

Daftar Mahasiswa

Nama Mahasiswa	NIM	Program Studi
Ajiz	12345	Hukum
Messi	54312	Mesin
Hudi	09876	Sipil
Ronaldo	31245	Arsitektur

Pilih Algoritma Sorting

Algoritma :

Bubble Sort

Mulai Sorting

Reset Visualisasi

Visualisasi Proses Sorting

```

=== BUBBLE SORT ===
Data awal : [Ajiz, Messi, Hudi, Ronaldo]

Pass 1 : [Ajiz, Hudi, Messi, Ronaldo]
Pass 2 : [Ajiz, Hudi, Messi, Ronaldo]

Hasil Akhir : [Ajiz, Hudi, Messi, Ronaldo]

```

Gambar 8

DAFTAR PUSTAKA

- [1] Modul Praktikum Struktur Data Pekan 7
- [2] Data Structures and Algorithms in Java